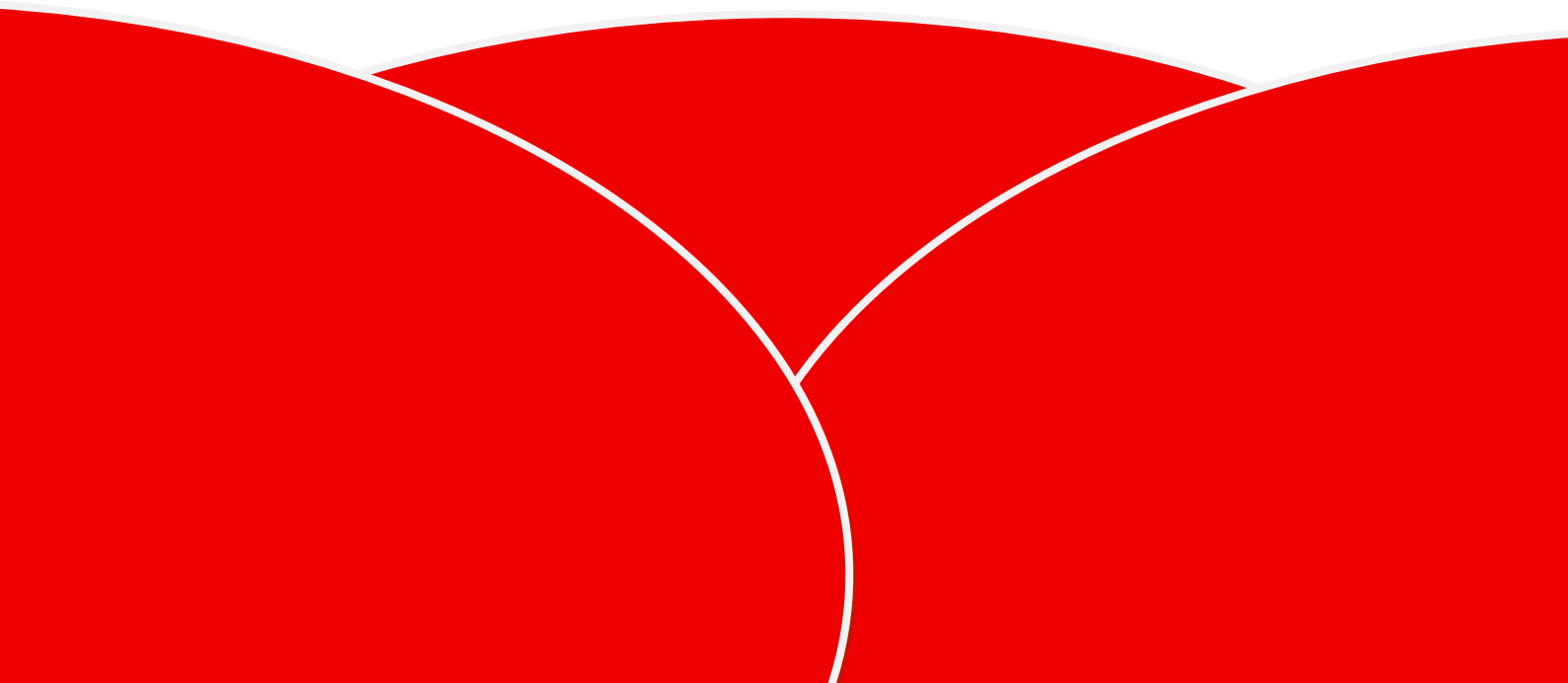


# Technisch Ontwerp 2.0

DIPPER



# Inhoudsopgave

|     |                                           |   |
|-----|-------------------------------------------|---|
| 1   | Inleiding .....                           | 3 |
| 2   | Functionele doelstellingen .....          | 3 |
| 2.1 | Dynamisch tonen van grafieken .....       | 3 |
| 2.2 | Configuratie via de webapp .....          | 4 |
| 2.3 | Automatisch updaten bij nieuwe data ..... | 4 |
| 2.4 | Filters toepassen op projectdata .....    | 4 |
| 2.5 | Exporteren van data .....                 | 4 |
| 3   | Gekozen talen en architectuur .....       | 5 |
| 3.1 | Backend .....                             | 5 |
| 3.2 | Front-end .....                           | 5 |
| 4   | GraphQL Implementatie .....               | 6 |
| 4.1 | Waarom GraphQL .....                      | 6 |
| 4.2 | Waarom Graphene .....                     | 6 |
| 5   | Dynamisch dashboard ontwerp .....         | 7 |
| 6   | Filtersysteem .....                       | 7 |
| 7   | Databaseontwerp voor grafieken .....      | 8 |
| 8   | Beveiliging .....                         | 8 |
| 9   | Conclusie .....                           | 9 |

# 1 Inleiding

DIPPER is een webapplicatie die door DI-Lab wordt gebruikt om projecten, studenten en bijbehorende documentatie centraal vast te leggen en te beheren. Binnen deze applicatie worden grote hoeveelheden projectdata verzameld, zoals voortgang, betrokken personen, GitHub activiteiten en andere relevante informatie. Om deze data overzichtelijk en inzichtelijk te maken, is een goed functionerend dashboard essentieel.

Voor deze opdracht is het doel om binnen DIPPER een dynamisch dashboard te ontwikkelen dat volledig vanuit de webapplicatie zelf kan worden geconfigureerd. Dit betekent dat grafieken, overzichten en filters niet hard in de back end van de applicatie worden vastgelegd, maar worden aangestuurd door data en instellingen uit de database. Gebruikers moeten in staat zijn om nieuwe grafieken toe te voegen, bestaande grafieken aan te passen en filters toe te passen zonder dat hier programmeerwerk voor nodig is.

## 2 Functionele doelstellingen

Het dynamische dashboard binnen DIPPER heeft als doel om projectgegevens van DI-Lab overzichtelijk, flexibel en zonder handmatige codewijzigingen inzichtelijk te maken. De belangrijkste functionele doelstellingen worden hieronder toegelicht.

### 2.1 Dynamisch tonen van grafieken

Het dashboard moet grafieken automatisch genereren op basis van data uit de database via GraphQL. De grafieken mogen niet hard-gecodeerd zijn in HTML of JavaScript.

Voorbeeld: Wanneer een nieuwe projectcategorie wordt toegevoegd in DIPPER, verschijnt deze automatisch als nieuwe datalijn in een bestaande grafiek zonder dat de frontendcode aangepast hoeft te worden.

## 2.2 Configuratie via de webapp

Beheerders moeten binnen de webapp zelf kunnen instellen:

- Welke grafieken zichtbaar zijn
- Welke data gebruikt wordt
- Welke labels en kleuren worden toegepast

Deze instellingen worden opgeslagen in de database en via GraphQL opgehaald door de frontend.

Voorbeeld: Een beheerder kan in de webinterface een grafiek “Projecten per student” toevoegen en instellen dat deze blauw moet zijn en standaard zichtbaar is. De frontend toont deze grafiek automatisch.

## 2.3 Automatisch updaten bij nieuwe data

Wanneer projectdata in DIPPER wordt aangepast of nieuwe projecten worden toegevoegd, moet het dashboard deze wijzigingen direct verwerken zonder herladen van code.

Voorbeeld:

Wanneer een nieuw GitHub project wordt gekoppeld aan DIPPER, wordt dit automatisch meegenomen in alle relevante grafieken bij het eerstvolgende dashboard refresh.

## 2.4 Filters toepassen op projectdata

Gebruikers moeten grafiekdata kunnen filteren op basis van eigenschappen zoals:

- Studenten
- Projectnamen
- Status
- Periode

Dit gebeurt via het filtersysteem dat via GraphQL wordt toegepast op de data.

Voorbeeld:

Een gebruiker kan filteren op “Student = Omar” en krijgt alleen projecten te zien waaraan Omar heeft gewerkt.

## 2.5 Exporteren van data

De data die in grafieken wordt weergegeven moet geëxporteerd kunnen worden naar verschillende formaten.

Ondersteunde formaten:

- CSV
- Excel (XLSX)

Voorbeeld:

Een docent kan een grafiek “Projectvoortgang” exporteren naar Excel om deze in een rapportage te gebruiken.

## 3 Gekozen talen en architectuur

### 3.1 Backend

**Python:** Wordt gebruikt als hoofdprogrammeertaal voor de backend logica en dataverwerking.

**Flask:** Lichtgewicht webframework dat de webapplicatie en API-endpoints verzorgt.

**Graphene (GraphQL):** Implementeert de GraphQL-laag waarmee de frontend flexibel data kan opvragen.

**MongoDB (PyMongo):** NoSQL database waarin projectdata en dashboardconfiguraties worden opgeslagen en opgehaald.

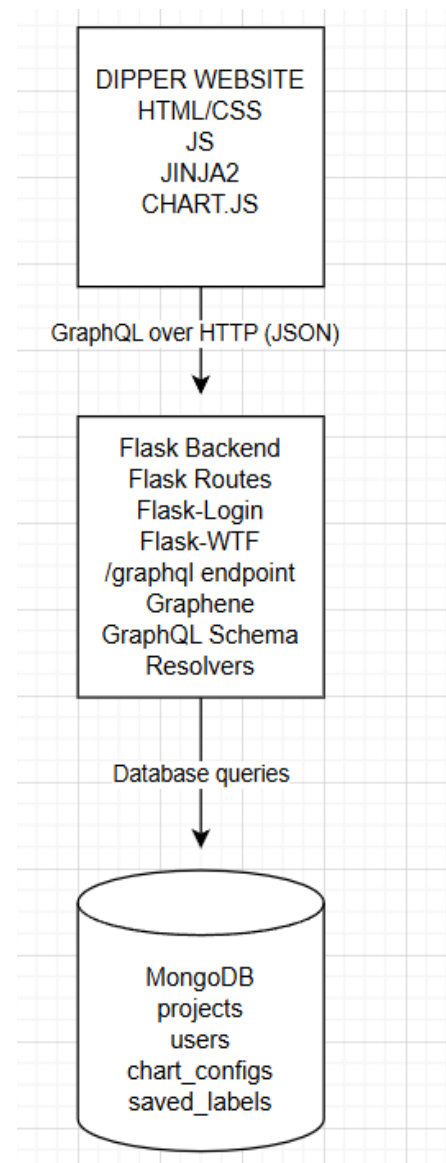
### 3.2 Front-end

**HTML:** Bepaalt de structuur van de webpagina's en het dashboard.

**CSS:** Verzorgt de vormgeving en layout van de gebruikersinterface

**JavaScript:** Maakt het dashboard interactief en verwerkt de GraphQL-responses.

**Chart.js:** Wordt gebruikt om grafieken en visualisaties te renderen op basis van de opgehaalde data.



## 4 GraphQL Implementatie

### 4.1 Waarom GraphQL

Voor het DIPPER-dashboard is gekozen voor GraphQL omdat het een flexibele en efficiënte manier biedt om data op te vragen. In tegenstelling tot traditionele REST API's kan de front-end bij GraphQL exact aangeven welke velden nodig zijn, waardoor alleen de benodigde data wordt opgehaald. Dit voorkomt over fetching, waarbij onnodige data wordt meegestuurd die niet in het dashboard wordt gebruikt.

Voor een dynamisch dashboard is dit essentieel, omdat verschillende grafieken verschillende datasets nodig hebben. Door GraphQL te gebruiken kan elke grafiek precies de juiste project, student of voortgangsgegevens opvragen zonder aparte API endpoints te hoeven maken. Hierdoor blijft de backend overzichtelijk en schaalbaar, terwijl de front-end maximale vrijheid behoudt.

### 4.2 Waarom Graphene

Voor de implementatie van GraphQL in de backend is gekozen voor Graphene, een GraphQL library voor Python die goed samenwerkt met Flask. Graphene maakt het mogelijk om op een gestructureerde manier GraphQL schema's, types en resolvers te definiëren, waardoor de data vanuit MongoDB eenvoudig beschikbaar kan worden gemaakt voor de front-end.

Tijdens de ontwikkeling is ook Ariadne overwogen, maar deze library bleek moeilijk te integreren in de bestaande Flask architectuur en gaf problemen bij het verwerken van schema's en requests. Graphene werkte daarentegen stabiel binnen de huidige backend structuur en bood een duidelijke koppeling tussen de database en de GraphQL query's. Daarom is Graphene gekozen als de meest betrouwbare en onderhoudsvriendelijke oplossing voor dit project.

## 5 Dynamisch dashboard ontwerp

Het dashboard binnen DIPPER is volledig dynamisch opgebouwd. Dit betekent dat grafieken, overzichten en visualisaties niet vast in de code zijn geprogrammeerd, maar worden gegenereerd op basis van configuraties die in de database zijn opgeslagen.

De backend bepaalt via configuratiedata:

- Welke grafieken bestaan
- Welke data gebruikt wordt
- Welke labels, kleuren en zichtbaarheid gelden

De frontend vraagt deze configuratie op via GraphQL en genereert vervolgens automatisch de juiste grafieken met behulp van JavaScript en Chart.js.

Hierdoor kunnen nieuwe grafieken of aanpassingen worden doorgevoerd via de webinterface zonder dat een ontwikkelaar de code hoeft te wijzigen. Dit maakt het dashboard flexibel, onderhoudsvriendelijk en geschikt voor toekomstig gebruik binnen DI-Lab.

## 6 Filtersysteem

Om gebruikers de mogelijkheid te geven om specifieke informatie te analyseren, bevat bij het maken van de dashboard een filtersysteem. Hiermee kan data worden beperkt op basis van eigenschappen zoals studenten, projecten, statussen of tijdsperiodes.

De filters worden via GraphQL toegepast op de databasequery's, waardoor alleen relevante gegevens worden teruggestuurd naar de frontend en in de grafieken worden verwerkt.

Binnen DIPPER zijn twee filterontwerpen uitgewerkt:

### Optie 1 (advanced):

Automatische verwerking van invoer, waarbij filters op basis van patronen zoals komma's of tekens worden gesplitst.

### Optie 2 (simple):

Een gebruikersvriendelijk systeem waarbij de gebruiker expliciet kiest hoe de invoer wordt verwerkt (bijvoorbeeld splitsen op komma of streepje).

Zonder Maus

①  ☒

②  ☒  
 ☒  
→ Split comma  
→ Split -  
→ Capitalize  
→ lowercase

Optie 2 is veiliger en beter beheersbaar, omdat invoer makkelijker kan worden gevalideerd en gesanitiseerd. Optie 1 biedt meer flexibiliteit, maar brengt hogere beveiligingsrisico's met zich mee, zoals injectieaanvallen.

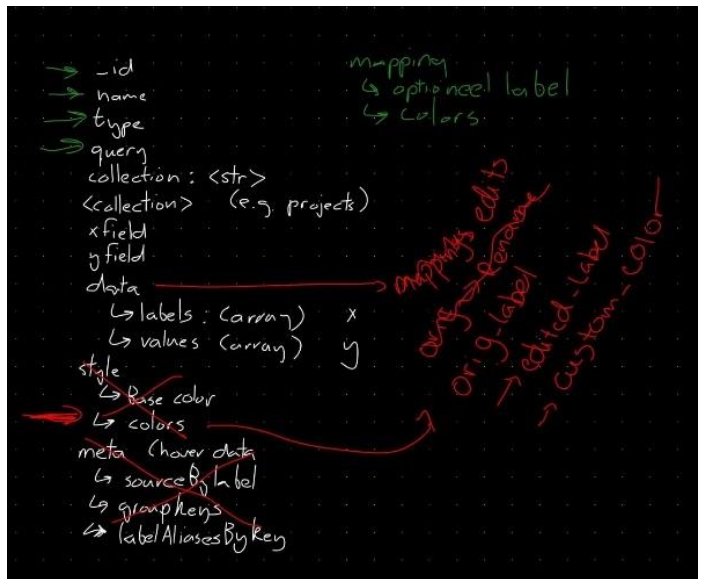
## 7 Databaseontwerp voor grafieken

In de huidige versie van het prototype voor DIPPER bevat de database veel opgeslagen statistieken en tussenresultaten. Dit maakt het systeem onoverzichtelijk en moeilijk te onderhouden.

In het nieuwe ontwerp wordt dit vereenvoudigd. In plaats van berekende data op te slaan, worden alleen grafiekconfiguraties bewaard, zoals:

- Grafieknaam
- De query
- Kleuren
- Zichtbaarheid

De daadwerkelijke data wordt altijd live opgehaald via GraphQL query's. Hierdoor blijft de database schoon, actueel en consistent met de onderliggende projectgegevens.



## 8 Beveiliging

Omdat het dashboard gebruikmaakt van gebruikersinvoer en GraphQL filters, is beveiliging een belangrijk onderdeel van het ontwerp.

Er wordt gebruikgemaakt van:

- Invoervalidatie
- Server-side filtering
- Beperking van GraphQL query's
- Bescherming tegen NoSQL injectie

Door deze maatregelen wordt voorkomen dat kwaadwillende gebruikers via filters of queries ongewenste databaseacties kunnen uitvoeren of gevoelige gegevens kunnen uitlezen.



## 9 Conclusie

Door gebruik te maken van GraphQL, een dynamische grafiekstructuur en een flexibele databaseopzet is het DIPPER dashboard toekomstbestendig en eenvoudig uitbreidbaar. Gebruikers kunnen zelfstandig nieuwe grafieken en filters instellen, terwijl de backend veilig, overzichtelijk en schaalbaar blijft. Dit ontwerp zorgt ervoor dat DI-Lab op een efficiënte en veilige manier inzicht krijgt in projectdata, zonder afhankelijk te zijn van handmatige codeaanpassingen.